AIAA 2003–1070

ICEG2D: A Software Package for Ice Accretion Prediction

David S. Thompson and Bharat K. Soni
Mississippi State University, Mississippi State, MS 39762

41st AIAA Aerospace Sciences Meeting and Exhibit
January 6–9, 2003/Reno, NV

For permission to copy or republish, contact the copyright owner named on the first page. For AIAA-held copyright, write to AIAA, Permissions Department, 1801 Alexander Bell Drive, Suite 500, Reston, VA 20191–4344

Prediction

David S. Thompson* and Bharat K. Soni†
Mississippi State University, Mississippi State, MS 39762

In this paper we describe the technology employed in the development of the integrated software package ICEG2D–a Navier-Stokes based ice accretion prediction code. ICEG2D provides a framework to couple the execution of its component modules with the ice accretion prediction code LEWICE and the flow solvers NPARC and WIND. The development philosophy was to rely on validated software whenever possible and implement techniques that required minimal changes to LEWICE, NPARC, and WIND. ICEG2D was designed to automatically perform five main functions: 1) generate a grid-ready surface definition based on the geometrical characteristics of the iced airfoil surface, 2) generate a high-quality structured grid starting from a surface definition, 3) generate the solver-dependent input files (for NPARC or WIND), 4) generate the input needed for the ice accretion prediction code LEWICE, and 5) generate a surface point distribution, grid, and NPARC or WIND initial/restart files in response to the deforming airfoil definition. The results generated using ICEG2D are compared with experimental data for representative ice shapes.

Introduction

PREDICTION of ice accretion on a full aircraft configuration and the resulting performance degradation is a long-term objective of icing research. However, many issues need to be addressed before routine simulation of full configurations is a reality. In spite of these recognized shortcomings, recent strides made in computational fluid dynamics (CFD) and increased availability of compute cycles are the genesis for expanded interest in simulations of icing and icing-related phenomena.

As implied in the previous paragraph, applications of computational methods to icing problems can be broadly categorized as being in one of two groups:[1]

1. Ice accretion–Given the flight and meteorological conditions, what is the resulting ice shape?

2. Icing effects–Given an ice shape, what is the flow field and the resulting performance degradation?

From a geometry/grid standpoint, the needs for the two applications are somewhat different.

For an ice accretion prediction, the geometry description and grid need to change in response to the ice accretion on the airfoil. In an icing effects analysis, the ice shape is fixed and only one surface discretization and interior grid are needed. Customized software such as SmaggIce[2,3] and the multiblock grid generator of Shih, et al.[4] have been developed specifically to perform these tasks. In this paper, we focus on the development of an automated system to perform ice accretion studies.

Currently, to perform an ice accretion analysis, the NASA Glenn ice accretion code LEWICE[5] uses a potential flow solution to estimate water droplet trajectories and collection efficiencies. The inviscid flow solution is also used with a boundary layer procedure to estimate surface heat transfer coefficients. These quantities are then used in an icing model to predict ice growth on the surface. The process is repeated while updating the airfoil shape with the accreting ice until the desired icing time is reached. For many conditions, this approach has proven adequate. However, for circumstances in which viscous effects are significant, e.g., complex ice shapes or high angle of attack flight conditions, a grid-based Navier-Stokes solution may be needed.

The primary difficulty associated with incorporating a grid-based flow solver with LEWICE is that, for the ice accretion problem, the effective airfoil shape and, consequently, the grid change as ice is accreted. Therefore, techniques are needed to perform point distribution on a curve (the airfoil surface definition) and generate a grid in the interior of the computational domain. Since these operations will need to be performed many times during an extended time icing case, these techniques necessarily require automation. Further, a strategy for generating an initial flow solution on each new mesh is needed.

We have completed a three-year effort to develop an automated, Navier-Stokes based ice accretion prediction package using the ice accretion prediction code LEWICE[5] and the flow solvers NPARC[6] and WIND.[7] The resulting software package, ICEG2D, is described in detail in three NASA Contractor Reports.[8–10] Our

*Associate Professor, Department of Aerospace Engineering, Computational Simulation and Design Center, ERC, Mississippi State University, Member AIAA.

†Currently Professor and Chair, Department of Mechanical Engineering, The University of Alabama at Birmingham, Associate Fellow AIAA.

Copyright © 2003 by the American Institute of Aeronautics and Astronautics, Inc. All rights reserved.

ware whenever possible and implement techniques that required minimal changes to LEWICE, NPARC, and WIND. Further, we exploited many of the existing WIND support modules[11–13] when appropriate. Our intent here is to provide salient information regarding the components of the software that provide geometry modeling, grid generation, and coupling with LEWICE.

This paper is organized as follows. We first give an overview of the approach we have taken to solving the ice accretion problem by coupling LEWICE with NPARC or WIND. We next describe several of the component modules in detail. We then describe the ICEG2D framework. We demonstrate the effectiveness of our approach by comparing ice shapes computed using ICEG2D with those predicted using LEWICE and those obtained from testing. Finally, we present conclusions discussing the relative merits of the approach.

Solution Strategy

The problem we are trying to solve is essentially one in which the body deforms as a function of time. The approach that has the potential to be the most accurate would be to compute a coupled flow field/droplet trajectory/ice growth procedure in a time accurate manner. However, this requires a significant investment of resources to validate each component of the procedure. Because of the extensive validation studies that have been performed using LEWICE, our development philosophy was to make minimal modifications to the LEWICE code itself. Therefore, we chose to exploit existing capabilities within LEWICE whenever possible. The minimum required modification to LEWICE is to include the capability to output a detailed surface definition after the ice accretion step. Similarly, the flow solvers, NPARC and WIND, were selected because they have been extensively validated.

Currently, LEWICE generates an ice shape by performing a panel method solution to generate the "inviscid" flow field. This data is then employed for a droplet trajectory analysis to provide collection efficiencies and impingement limits. The flow field data is also used as input to a boundary layer routine to compute the surface heat transfer. These data are then input to an icing model that predicts a new ice shape and the process is repeated.

The Navier-Stokes solution data can be included in two components of the icing prediction process. It can be used to replace the panel method solution thereby affecting the droplet trajectory computation and the boundary layer computation through the specification of the edge conditions. We denote this degree of coupling Level 0. It can also be used to replace the boundary layer heat transfer calculation. We denote this degree of coupling Level 1. We denote the fully-coupled flow field/droplet trajectory/ice growth

Alone, Level 0 offers potential to improve ice accretion prediction for more complex ice shapes. Viscous effects for flows around glaze ice shapes with significant horn accretions may be important and cause the panel method flow field to be significantly in error. Level 0 coupling can be achieved within LEWICE by setting the variable IGRID=1 and providing input to LEWICE in the form of a grid file and a solution file. Level 1 coupling includes Level 0 coupling as well as a surface heat transfer computation from the viscous flow field. Level 1 coupling may be achieved by setting the variables IHTC=1 and IGRID=1 and providing input to LEWICE in the form of a file containing the surface convective heat transfer coefficients in addition to the grid and solution files. The obvious additional advantage of the Level 1 approach is the removal of boundary layer assumptions needed for the heat transfer calculation which may not be valid for complex viscous flows with separated regions. We now consider the technologies used in several of the ICEG2D component modules.

Geometry Modeling

The general philosophy we have employed for geometry modeling is to generate a discrete approximation to the surface that is as faithful as possible for a given number of points. Points are clustered in regions of high surface curvature to ensure geometric fidelity of the iced airfoil surface definition. The user is allowed to specify the maximum and minimum spacing permitted on the airfoil surface. At no time during the process is any explicit smoothing applied to the airfoil surface definition. However, as discussed below, the airfoil surface is represented internally as a NURBS curve. When the points are distributed on the surface, some smoothing is present due to the fact that the NURBS representation, as implemented, does not reproduce slope discontinuities in the surface description. However, this smoothing is present once a NURBS representation is used to describe the geometry and is not a function of the distribution algorithm.

Internal Surface Description

Given a valid description of the iced airfoil surface, an internal representation of the surface must be generated. Routines from the geometry/grid library GGTK[14] are used to define a NURBS representation for the iced airfoil surface. The parametric description of the NURBS curve is given by

$$\mathbf{P}_N(u) = (x(u), y(u)) \tag{1}$$

where u is the parametric variable. An inversion routine from GGTK is used to compute the control points needed to produce the specified surface.

The distribution of points on the airfoil is based on the equidistribution principle.[15] The approach used here is to distribute points on the NURBS description of the iced region of the airfoil, $\mathbf{P}_N(u)$, using a weight function based on surface curvature. For a distribution of N points along a curve of length s_{iced}, the location of the ith point is computed using equidistribution from

$$\int_0^{s_i} w(s)\,ds = \frac{i-1}{N-1}\int_0^{s_{iced}} w(s)\,ds \tag{2}$$

where $w(s)$ is the weight function and s is the arc length along the curve.[15] The objective is to determine the value of the integrand s_i that satisfies the equality for each value of i. A direct result of using the equidistribution formulation is that in regions where the positive weight function is large, the grid spacing is correspondingly small. Therefore, if the weight function is based on surface curvature, points will be clustered in regions of higher curvature leading to a better representation of the surface.

Weight Function Definition

Using appropriate GGTK routines, the surface curvature is computed using

$$K(u) = \frac{\left|\left(\mathbf{P}'_N\cdot\mathbf{P}'_N\right)\mathbf{P}''_N - \left(\mathbf{P}''_N\cdot\mathbf{P}'_N\right)\mathbf{P}'_N\right|}{\left(\mathbf{P}'_N\cdot\mathbf{P}'_N\right)^2} \tag{3}$$

where it is understood that $\mathbf{P}_N$ and its derivatives are functions of the parameter u. A clustering function is now defined at each of the integration nodes

$$c(u) = K(u)^{\left(1-\frac{1}{\alpha}\right)} \tag{4}$$

where $K(u)$ is the curvature and $\alpha \geq 1$ is a clustering parameter. The general effect of decreasing α is to increase the strength of the peaks of the normalized curvature, i.e., to increase the variation in curvature and increase the clustering in the point distribution. The weight function is then smoothed. It is important to note that the geometry is not being smoothed—only the clustering function used to define the weight function in the equidistribution scheme is being smoothed. The purpose of the smoothing is to reduce the variation of the surface point distribution in regions of rapid variation in curvature to produce a smooth surface grid.

Finally, the normalized weight function is defined in terms of the smoothed clustering function using

$$w(u) = \frac{\overline{c}(u) - \overline{c}_{min}}{\overline{c}_{max} - \overline{c}_{min}} \times (w_{max} - w_{min}) + w_{min} \tag{5}$$

where w_{min} and w_{max} are the minimum and maximum allowable values of the weight function, respectively. Usually, w_{max} is taken to be unity and, by equidistribution,

$$w\Delta s = C \tag{6}$$

$$w_{max}\Delta s_{min} = w_{min}\Delta s_{max} = C \tag{7}$$

where

$$C = \frac{1}{N-1}\int_0^{s_{iced}} w(s)\,ds. \tag{8}$$

Taken together, these equations imply that

$$w_{min} = w_{max}\frac{\Delta s_{min}}{\Delta s_{max}} \tag{9}$$

and that

$$N = 1 + \frac{1}{w_{max}\Delta s_{min}}\int_0^{s_{iced}} w(s)\,ds. \tag{10}$$

Therefore, for a given airfoil geometry, i.e., a given $w(s)$ with $w_{max} = 1$, the minimum and maximum spacings may be specified and the number of points computed. Although not discussed here, local spacing control may be obtained by manipulation of the weight functions.

Implementation

For the problem at hand, the weight function is known explicitly in terms of the parametric arc length u. Therefore, using a change of variable, the equidistribution relation becomes

$$\begin{gathered}\int_0^{u_i} w(u)\left(\tfrac{ds}{du}\right)du = \\ \tfrac{i-1}{N-1}\int_0^1 w(u)\left(\tfrac{ds}{du}\right)du\end{gathered} \tag{11}$$

where

$$\frac{ds}{du} = \sqrt{\mathbf{P}'_N(u)\cdot\mathbf{P}'_N(u)} \tag{12}$$

and $\mathbf{P}'_N(u)$ is computed using appropriate GGTK routines. The integral is evaluated by specifying the integration nodes on the NURBS representation $\mathbf{P}_N(u)$ in parametric space and performing a piecewise linear fit of the integrand. Using the integral formulation eliminates many of the problems associated with the finite-difference version of the equidistribution equation. In particular, no iteration is required to obtain the point distribution. Routines from GGTK are then used to evaluate the resulting (x_i, y_i) locations on $\mathbf{P}_N(u_i)$.

Grid Generation

Automated structured grid generation for complex configurations is an, as yet, unsolved challenge. Even grids for the topologically simple single-element iced airfoils are challenging because of concave regions occurring on the surface of the airfoil. These regions pose problems for algebraic or elliptic grid generation algorithms because of the difficulty in appropriately placing corresponding points on the outer boundary of the grid. One approach that obviates this problem is to use a marching technique to generate the

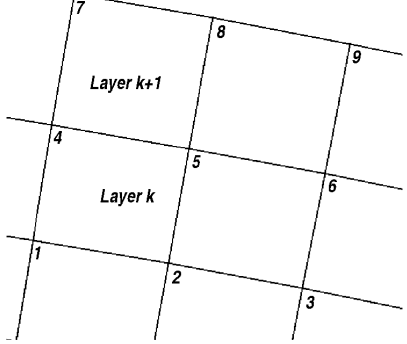


Fig. 1 Reference grid definition

grid.[16–20] In this context, a marching scheme uses the surface distribution as the initial data curve and generates a grid layer by layer until a specified number of layers have been generated. Note that here, unlike other parabolic methods, no attempt is made to march to a specified outer boundary. Since the basic grid generation algorithm has previously been presented,[19,20] only a cursory description of it follows.

Parabolic Grid Generation

In the parabolic method,[16,17,19,20] a reference grid is utilized to make the marching problem well posed. Starting from the initial data surface, points 1, 2, and 3 in Fig. 1, two layers of a reference grid are generated. The Poisson grid generation equations are then applied to points on the intermediate surface of the reference grid, points 4, 5, and 6 in Fig. 1, thereby smoothing the reference grid. The outer surface of the reference grid, points 7, 8, and 9 in Fig. 1, is updated after each iteration of the Poisson grid equations. The outer surface of the reference grid is discarded once the desired number of smoothing iterations is complete. It should be noted that the resulting smoothed grid still exhibits many of the characteristics of the reference grid.

As is customary,[15] a transformation of the form

$$\begin{aligned} \xi &= \xi\left(x, y\right) \\ \eta &= \eta\left(x, y\right) \end{aligned} \tag{13}$$

is defined and the inverse transformation

$$\begin{aligned} x &= x\left(\xi, \eta\right) \\ y &= y\left(\xi, \eta\right) \end{aligned} \tag{14}$$

is assumed to exist. The initial data surface is assumed to be defined by a η=constant surface so that η is the marching direction.

Reference Grid Definition

The reference grid is generated algebraically to be locally orthogonal. The intermediate surface of the

$$\mathbf{r}_{i,1}^{j} = \mathbf{r}_{i,0}^{j} + \delta_i^j \mathbf{n}_{i,0}^{j} \tag{15}$$

where $\mathbf{r}_{i,0}^{j}$ is the position vector to the point located at i on the initial data surface (0 subscript), $\mathbf{r}_{i,1}^{j}$ is the position vector to the point i on the first surface of the reference grid, δ_i^j is the specified distance distribution, and $\mathbf{n}_{i,0}^{j}$ is the unit surface normal. The outer surface of the reference grid in layer j is generated using

$$\mathbf{r}_{i,2}^{j} = \mathbf{r}_{i,1}^{j} + \delta_i^{j+1} \mathbf{n}_{i,1}^{j} \ . \tag{16}$$

The normal distance distribution is generated using a hyperbolic tangent stretching function.[21] The distance to the first point off the wall is specified along with the distance to the last point and the total number of points.

Reference Grid Smoothing

The equations used to smooth the reference grid are the standard Poisson equations[15] used in grid generation with the assumption of grid orthogonality, i.e., g_{12} is zero:

$$g_{22}\left(\mathbf{r}_{\xi\xi} + \phi \mathbf{r}_{\xi}\right) + g_{11}\left(\mathbf{r}_{\eta\eta} + \psi \mathbf{r}_{\eta}\right) \tag{17}$$

where

$$\begin{aligned} g_{11} &= x_{\xi}^2 + y_{\xi}^2 \\ g_{22} &= x_{\eta}^2 + y_{\eta}^2 \end{aligned} \ . \tag{18}$$

The partial derivatives in the Poisson equation are approximated using second-order central differences. The resulting system of equations is solved using line relaxation.

The form of the control functions plays an important role in determining the quality of the grid. The form of the control functions used here does not include curvature effects[22] and is given by

$$\begin{aligned} \phi &= -\tfrac{1}{2}\tfrac{1}{g_{11}}\tfrac{\partial g_{11}}{\partial \xi} \\ \psi &= -\tfrac{1}{2}\tfrac{1}{g_{22}}\tfrac{\partial g_{22}}{\partial \eta} \end{aligned} \ . \tag{19}$$

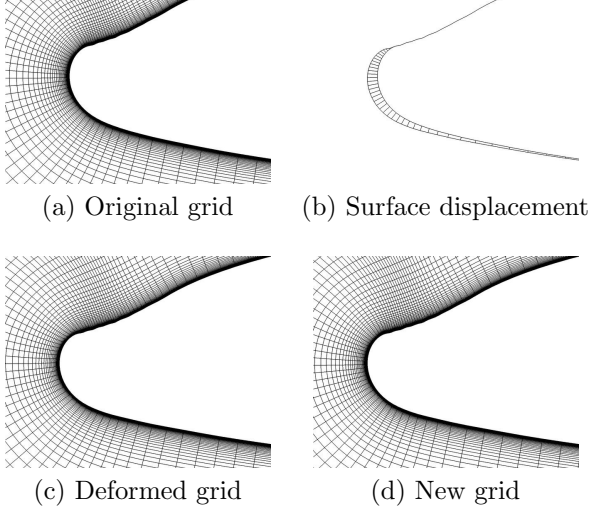
It should be noted that Equation 19 is equivalent to employing the "geometric" weight functions defined by

$$\begin{aligned} w_g^{\xi} &= \tfrac{1}{\sqrt{g_{11}}} \\ w_g^{\eta} &= \tfrac{1}{\sqrt{g_{22}}} \end{aligned} \tag{20}$$

with ϕ and ψ defined using

$$\begin{aligned} \phi &= \tfrac{1}{w_g^{\xi}}\tfrac{\partial w_g^{\xi}}{\partial \xi} \\ \psi &= \tfrac{1}{w_g^{\eta}}\tfrac{\partial w_g^{\eta}}{\partial \eta} \end{aligned} \ . \tag{21}$$

For structured grid generation, the control function ϕ is computed using the point distribution on the initial



(a) Original grid (b) Surface displacement

(c) Deformed grid (d) New grid

Fig. 2 Interpolation strategy

data line (g_{11} on the airfoil surface) and held constant along each ξ=constant line. ψ can be approximated using the normal distance distribution by noting that $g_{22} = \delta^2$.

Recall the objective here is not to solve the elliptic grid generation equations but rather use them as a smoothing mechanism. In practice, it has been found that reducing the grid point movement induced by the Poisson smoothing by two orders of magnitude results in a grid that is sufficiently smooth. Typically, this process requires the equivalent of fewer than ten iterations of an elliptic solver. Domains containing strongly non-convex regions require additional iteration. Grids generated using these techniques require only a matter of seconds on typical workstations.

Flow Solution Restart

We now describe a procedure for restarting the flow solution once an ice accretion cycle is complete. A relatively simple approach would be to restart the flow solver from a uniform flow. This approach is, however, computationally expensive because of the severe transients that the flow solver must dissipate. A more promising approach would be to interpolate the solution directly to the new mesh. However, this procedure is complicated by the fact that the mesh boundary has changed due to the ice accretion. Further complicating the issue is that the number of points employed to define the surface may change due to the ice accretion.

In the procedure we have developed to generate a restart solution for the Navier-Stokes solver, we start from the original grid as shown in Fig. 2(a). We then compute displacement vectors for each point on the airfoil surface as shown in Fig. 2(b). Once the displacement vectors are computed, the interior grid points are displaced according to the solution of Laplace's equation. Note that the deformed grid, shown in Fig. 2(c), has the same dimensions and is topologically equivalent to the original grid. The solution vector on the original grid is injected to the deformed grid using

$$q(\xi, \eta) = \text{constant} \tag{22}$$

where q is the solution vector. The new grid, shown in Fig. 2(d), is generated after a new surface is defined. The new grid, while it may not be topologically equivalent to either the original grid or the deformed grid, has the same bounding curves as the deformed grid. Therefore, a linear interpolation can be used to interpolate the solution from the deformed grid to the new grid.

Boundary Point Displacement

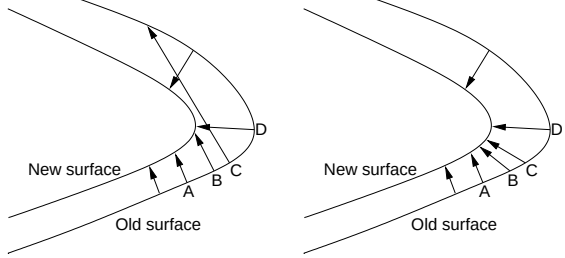
The first step of the restart procedure is to compute the boundary displacement due to ice accretion. LEWICE was modified to output a detailed airfoil (with ice accretion) definition at the conclusion of each icing cycle. Internally, LEWICE uses approximately 5000 points to define the iced airfoil shape. This resolution is much higher than the surface definition used for the Navier-Stokes solution (typically 300-400 surface points).

In general, at a given point on the old airfoil definition, we assume the displacement vector to be in the direction of the local surface normal. This approach is adequate for regions where the airfoil surface is convex. Fig. 3 shows a concave region of the airfoil surface. As shown in Fig. 3(a), this approach can be used without modification for points A and D. Point B presents a problem because the normal and the local tangent to the new curve are not close to perpendicular, i.e., the minimum angle between the normal and the tangent is less than a specified value. Point C presents a problem because the normal vector "crosses" the normal vector at D. The approach taken here is to tag points A and D as points where the normal displacement can be used. Points B and C are tagged and identified as a contiguous segment, bounded by points A and D, where the normal displacement vector is unacceptable. An arc length interpolation is then used to distribute the points on the new surface in a manner similar to the distribution along the segment $A - D$ on the old surface as shown in Fig. 3(b).

Field Point Displacement

Once the boundary point displacements are obtained, it is necessary to prescribe a deformation for the remainder of the grid. Essentially, we want to define a boundary value problem since the displacements in the interior of the domain should be determined by the displacements on the boundary. The simplest approach is to specify the field point positions as the solution of Laplace's equation in physical (x, y) space:

$$\begin{aligned} \frac{\partial^2 \mathbf{d}}{\partial x^2} + \frac{\partial^2 \mathbf{d}}{\partial y^2} = 0 \quad & \text{in } D \\ \mathbf{d} = \mathbf{d}_o \qquad & \text{on } \partial D \end{aligned} \tag{23}$$



(a) Normal vector (b) Modified vectors

Fig. 3 Displacement vector definition

where $\mathbf{d}$ is the displacement vector and $\mathbf{d_o}$ is the displacement vector on the boundary. The advantage of using this approach is the fact that the solution of Laplace's equation satisfies a maximum principle, i.e., the maximum displacements will occur on the boundary. However, this does not guarantee that a valid mesh will be generated. Poisson smoothing, Eq. 17, can be applied to the resulting grid to ensure that no "folded" cells are present. Since, after deformation, the grid is topologically equivalent to the grid before deformation (in computational space), the control functions can be evaluated on the original grid and used in the Poisson smoothing equation.

LEWICE Restart

An additional point that needs to be addressed is that LEWICE is not designed to be restarted. That is, LEWICE is not designed to perform a ten-minute ice accretion simulation in ten separate one-minute runs. In the icing model used by LEWICE, the first few minutes of icing are treated differently.[23] Therefore, LEWICE was modified to include a start time for each run. Differences with restarted results were also encountered due to round-off errors and the overall sensitivity of the icing problem to small changes in the geometry.

ICEG2D Overview

We now provide an overview of ICEG2D–the software package developed to automatically perform ice accretion studies using LEWICE[5] and the structured grid flow solvers NPARC[6] or WIND.[7] ICEG2D is based on a framework in which various operations are performed to generate the geometry, grid, and input files for the flow solver. The user interface to the ICEG2D framework is a C++ program that utilizes file associations and directives to perform the necessary functions. The various component modules are written in C and Fortran90. These component modules, as well as NPARC, WIND, and LEWICE, are executed as system commands.

The ICEG2D framework is based on a system of internal associations with default or user-prescribed file names. The primary association is the *case_name*. By defining the association *case_name*, a set of automatically defined file names is utilized by the framework for module input and output. These file names are defined using *case_name* and a default extension based on the file type. These associations may be changed using appropriate ICEG2D directives. Minimally, the user must provide the iced airfoil input file and two problem definition files (for the selected flow solver and LEWICE). Default values for all parameters are defined within ICEG2D using Fortran90 namelists. These default values have proven effective for the cases examined to date.

ICEG2D directives can be used to "define" the associations, to "generate" the surface distribution, grid, etc., and to perform selected miscellaneous functions. The directives all have the same form:

```
def case naca0012
def solver nparc
def info NPARC_input.inf
def input_grid naca0012.grd
def input_q fort.30
#   Generate surface distribution and grid
gen dist
gen grid
#   Generate NPARC input and restart files
gen input
gen restart
def output 411_minute0.out
#   Generate NPARC solution
gen solution
#   Generate LEWICE input including
#   flow field input
def lewice_input 411.inp
gen lewice_input
#   Ice accretion step #1
gen lewice
#   Deform grid and generate updated
#   restart file for interpolation
gen grid_update
gen restart_update
#   Generate new airfoil definition and grid
def iced_geom updated_airfoil
def dist 1-minute_shape.dst
gen dist
gen grid
#   Generate new NPARC input and restart
#   files (solution interpolation from
#   updated file)
gen input
gen restart
def output 411_minute1.out
#   Generate new NPARC solution and repeat
...
```

Table 1 Sample ICEG2D input stream using the flow solver NPARC

duces/acts on] [additional info (if needed)]

The vocabulary used by ICEG2D is described in detail in the NASA Contractor Report.[9] A sample listing of ICEG2D directives used to generate a single ice accretion step is shown in Table 1.

ICEG2D may be executed in batch mode or from a command line prompt. In batch mode, a text file containing a stream of ICEG2D directives is used as input. In the interactive mode, the user executes ICEG2D and issues directives from a command line within a Unix shell.

Results

We now present several representative comparisons of results predicted using ICEG2D with experimental data. The cases discussed below were selected from the LEWICE validation study[24] where they were denoted as cases 405 and 411. 405 is a seven-minute rime ice accretion while 411 is a 14-minute glaze ice accretion. In each case, an NACA0012 airfoil section was at an angle of attack of 3.5 degrees, the chord length was $0.5334m$, the flight speed was $102.8m/s$, and the static pressure was $100000N/m^2$. These conditions corresponded to a Mach number of 0.324 and a Reynolds number of 4.63×10^6 based on the freestream velocity. Additionally, for each case the relative humidity was 100%, the liquid water content was $0.55g/m^3$, and the median volume droplet was taken to be 20μ. The only difference was in the static temperature. In 405, the static temperature was $250.37K$ while in 411 the temperature was $262.04K$. All ice shapes were generated using multiple one-minute accretion cycles.

The solutions for the clean airfoil flow field were obtained on a grid with dimension 365×101. The grid had a C-type topology and extended 15 chords from the airfoil. The nondimensional distance to the first point off the wall was 1.0×10^{-5}. This corresponds to a y^+ value on the order of one. Initial NPARC solutions were obtained by running NPARC for 50000 iterations using the Spalart-Allmaras turbulence model. NPARC solutions were run on an SGI Challenge 10000 XL. On this machine, approximately one hour was needed to perform 2500 NPARC iterations. Initial WIND solutions were obtained by running WIND for 200 iterations (with 10 subiterations) also using the Spalart-Allmaras turbulence model. On the same machine, approximately 2 hours are needed to perform 2000 WIND subiterations. It should be noted that the WIND solutions converge much more rapidly than the NPARC solutions. However, it is difficult to compare the level of convergence between the two codes. The geometry, grid generation, and interpolation tasks required only a few minutes per cycle. All geometry definitions and grids were generated automatically using appropriate ICEG2D modules.

temperature boundary condition used for the flow field calculation. In all cases an isothermal wall boundary condition is employed. The question remains as to what temperature should be specified. An estimate for a lower bound is the isentropic stagnation temperature. A reasonable upper bound is the freezing point of water. Although not shown here, the computations made using the freezing point as the isothermal wall temperature showed showed significantly better agreement with experimental data than results predicted using other values for the wall temperature.

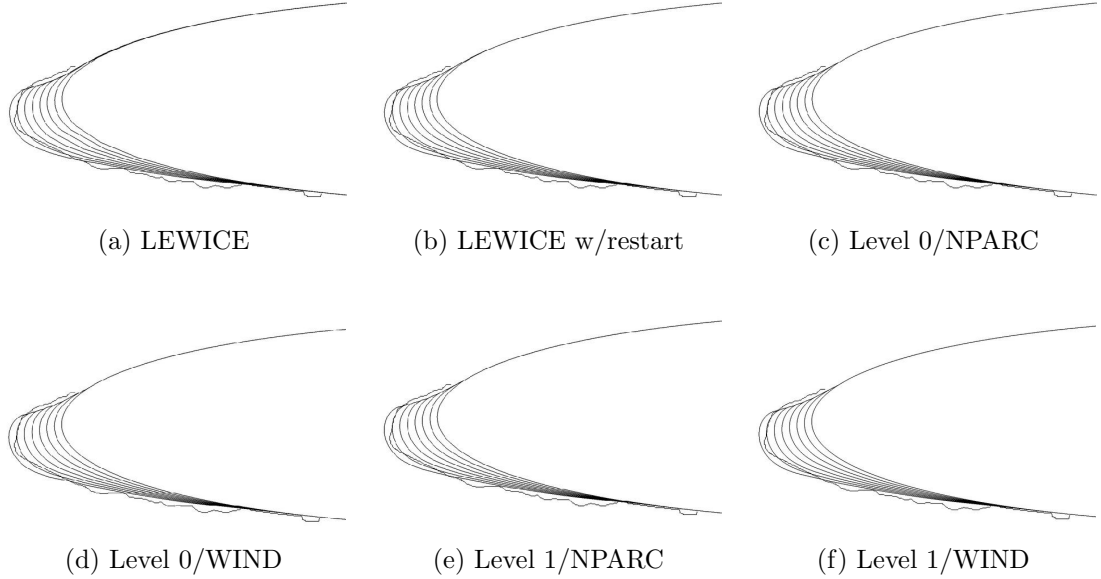
Case 405

The conditions for case 405 were used to generate ice accretions using the following techniques: standard LEWICE, LEWICE restart (to test the restart capabilities), ICEG2D Level 0/NPARC, ICEG2D Level 0/WIND, ICEG2D Level 1/NPARC, and ICEG2D Level 1/WIND. Figs. 4(a)-(f) show the predicted ice shapes for a seven-minute icing time (one minute increments) for each of these methods, respectively. Also shown is the experimentally determined ice shape. For this case, the differences in the predicted ice shapes are relatively small. In particular, no benefit was garnered from using the Level 0 or Level 1 coupling. Since the resulting ice shape is relatively simple, the flow field should be well predicted by the coupled panel method/boundary layer procedure. Further, for this "cold" condition, the ice accretion problem is nearly a deposition problem. This similarity between the ice shapes predicted using ICEG2D and LEWICE is consistent with Fig. 5 which shows a comparison between the collection efficiencies computed for the same ice shape, in this case the six-minute shape, using the flow fields from the inviscid flow solver and from WIND. Note that the differences between the two curves are insignificant.

Fig. 6 shows a plot of the seven-minute ice accretion history as well as the grid generated after the final accretion cycle for the Level 0/NPARC calculation.

Fig. 7 shows the NPARC convergence history for the ice accretion calculation. Recall that the first 50000 iterations are the baseline solution. Thereafter, the residual "spikes" every 5000 iterations in response to a change in the shape of the airfoil. Other cases with 2500 and 10000 NPARC iterations per one-minute accretion cycle did not show significant differences. Again, this can be attributed to the "cold" icing conditions.

Fig. 8 shows the WIND convergence history for the ice accretion calculation. Recall that the first 2000 subiterations are the baseline solution. Thereafter, the residual "spikes" every 1000 subiterations in response to a change in the shape of the airfoil.



(a) LEWICE (b) LEWICE w/restart (c) Level 0/NPARC

(d) Level 0/WIND (e) Level 1/NPARC (f) Level 1/WIND

Fig. 4 Case 405: Comparison of predicted shapes with experimental data

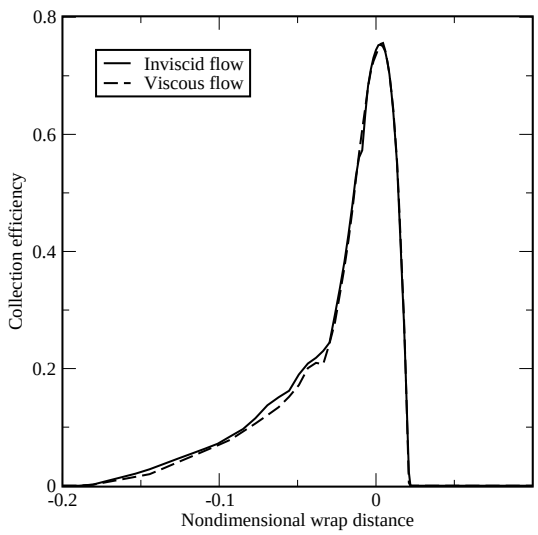


Fig. 5 Case 405: Collection efficiencies

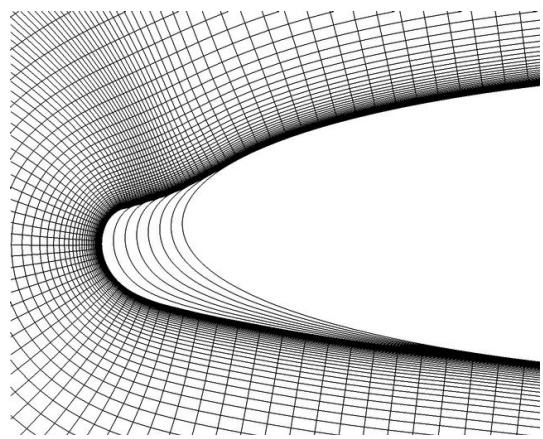


Fig. 6 Case 405: Accretion history and final grid

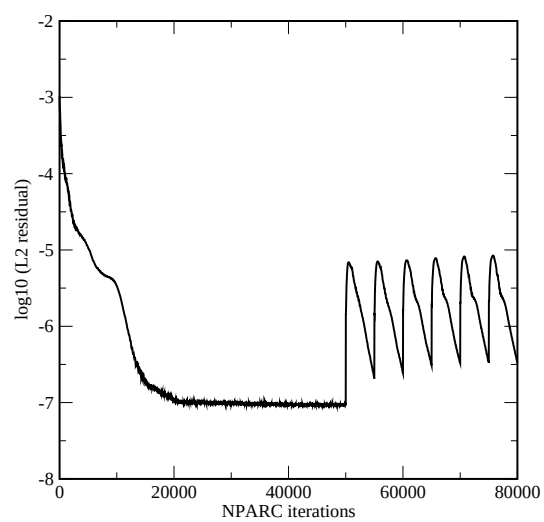


Fig. 7 Case 405: NPARC convergence history

Case 411

The conditions for case 411 were used to generate ice accretions using the following techniques: standard LEWICE, LEWICE restart, ICEG2D Level 0/NPARC, ICEG2D Level 0/WIND, ICEG2D Level 1/NPARC, and ICEG2D Level 1/WIND using the freezing point of water as the isothermal wall temperature. Figs. 9(a)-(f) show the predicted ice shapes for a 14-minute icing time (one minute increments) for each of these methods, respectively. In this case, it is important to note that the technique used makes a significant difference in the final shape. This is due to the "warmer" conditions. In this case, the aerodynamic effects play a larger role in the ice accretion process. First, we compare results predicted using standard LEWICE, shown in Fig. 9(a), with results

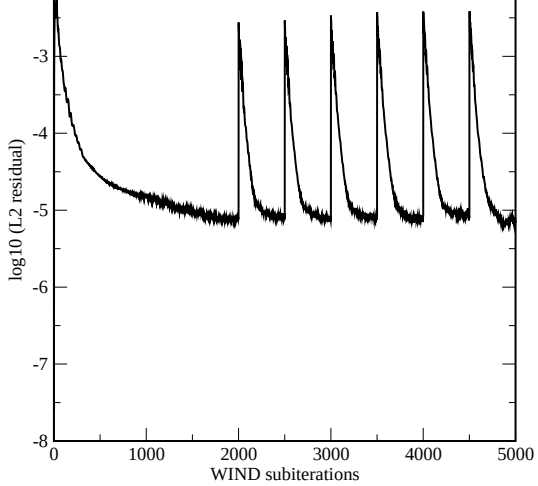


Fig. 8 Case 405: WIND convergence history

from the LEWICE with restart capability, shown in Fig. 9(b). Although the final shapes are quite similar, there are some differences most notably in the lower horn. It should be noted that we found LEWICE to be very sensitive to perturbations in the input geometry. In light of this, it is not surprising that there are some differences between the standard LEWICE and the LEWICE with restart capability.

We next examine the ICEG2D Level 0 predictions shown in Figs. 9(c) and 9(d). For this case, Level 0 coupling does a better job of capturing the general shape of the glaze ice accretion than LEWICE alone. Although there are differences between the ice shapes predicted using the NPARC and WIND Level 0 couplings, these differences are less significant than those that occur when these shapes are compared with the shapes predicted by LEWICE alone. For more complex ice shapes, such as the one under consideration, the viscous solution may be considerably different from the inviscid solution. We believe this to be the reason for the improved shape of the Level 0 predictions. It should be noted that somewhat less ice is accreted using the Level 0 coupling as opposed to the LEWICE calculation. Fig. 10 shows a comparison between the collection efficiencies computed for the same ice shape, in this case the 13-minute shape, using the flow fields from the inviscid flow solver and from WIND. Notice that while the shapes are similar, the levels are somewhat different.

Finally, we examine the results obtained from the Level 1 calculations shown in Figs. 9(e) and 9(f). We note that the agreement obtained using both Level 1 couplings is not comparable to that obtained using the other approaches. The gross shape is not well predicted and the horn angles are incorrect. In both cases, the amount of accreted ice is also underpredicted. We attribute these discrepancies to an inadequate representation of the surface roughness of the ice. In incorporated to account for the resulting enhanced heat transfer. Currently, no roughness effects were incorporated in the Navier-Stokes computations.

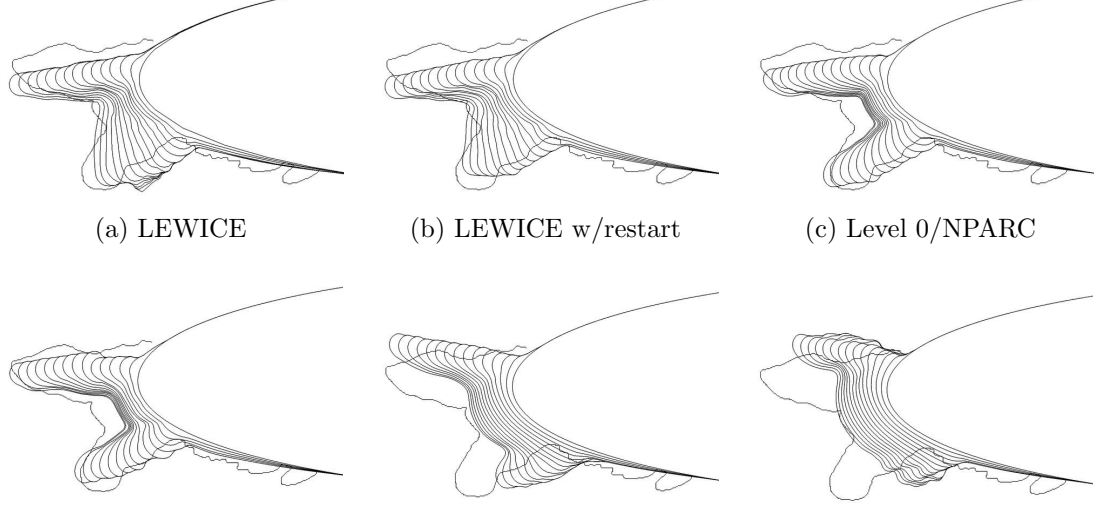
To illustrate the automatic geometry modeling and grid generation capabilities, Fig. 11 shows a plot of the 14-minute ice accretion history as well as the grid generated after the final accretion cycle. As can be anticipated from a grid generated by a marching approach, there is significant clustering of grid lines emanating from concave regions of the airfoil surface.

Fig. 12 shows the NPARC convergence history for the ice accretion calculation. Again, the first 50000 iterations are the baseline solution. Thereafter, the residual "spikes" every 10000 iterations in response to a change in the shape of the airfoil. Other cases with 2500 and 5000 NPARC iterations per one-minute accretion cycle showed significant differences. Based on our experiences, 10000 is the recommended minimum number of NPARC iterations for more complex ice shapes. In an attempt to reduce the overall CPU time, we tried to shorten the icing cycle time per step to 30 seconds and use 5000 NPARC iterations per icing cycle. Unfortunately, the 5000 iterations were not adequate to reduce the residual to acceptable levels. Although not shown here, we also considered the lift coefficient as an alternative measure of convergence of the NPARC solution. Again, we found that 10000 NPARC iterations were necessary for each icing cycle to generate a steady lift coefficient. Fig. 13 shows the convergence history for this case using WIND. In general, trends similar to those reported for NPARC were observed for WIND with 1000 subiterations being sufficient.

Concluding Remarks

An integrated software package, ICEG2D, has been developed to couple a viscous flow solver, in this case NPARC or WIND, with the ice accretion prediction code LEWICE. The motivation for this effort is an attempt to improve ice accretion prediction for those cases in which the panel method/boundary layer approach currently in use in LEWICE may prove inadequate. ICEG2D provides automated geometry modeling and grid generation for iced airfoils as well as generation of restart and input files for NPARC or WIND and LEWICE. ICEG2D provides two levels of coupling: Level 0 in which the viscous solution replaces the panel method but the boundary layer method is retained to compute surface heat transfer coefficients and Level 1 in which the viscous solution is also used to provide the surface heat transfer coefficient.

Comparison with results predicted using LEWICE and with experimentally obtained ice shapes show that: 1) for "cold" rime ice cases LEWICE performs adequately and neither Level 0 or Level 1 coupling improves the results and 2) for "warm" glaze ice cases



(a) LEWICE (b) LEWICE w/restart (c) Level 0/NPARC

(d) Level 0/WIND Level 1/NPARC (e) Level 1/WIND

Fig. 9 Case 411: Comparison of predicted shapes with experimental data

Level 0 coupling improves the resulting shape and Level 1 coupling degrades it. This issue remains under investigation, but we hypothesize that the problem is related to an inadequate representation of the effects of surface roughness in the flow field solution.

One point not addressed here is unsteadiness of the resulting flow. At present, a "snapshot" of the viscous flow field is used for all LEWICE calculations. A better approach might be to use time-averaged flow field data as input to LEWICE. This has not been investigated to date.

The real question, of course, is "Do the improvements in the predicted ice shape warrant the additional computational expense?" Based on our admittedly limited validation efforts, the results appear mixed. We considered several cases in addition to those included here. The results were fairly consistent. Further study is needed to determine if these "improvements" are significant in terms of performance degradation.

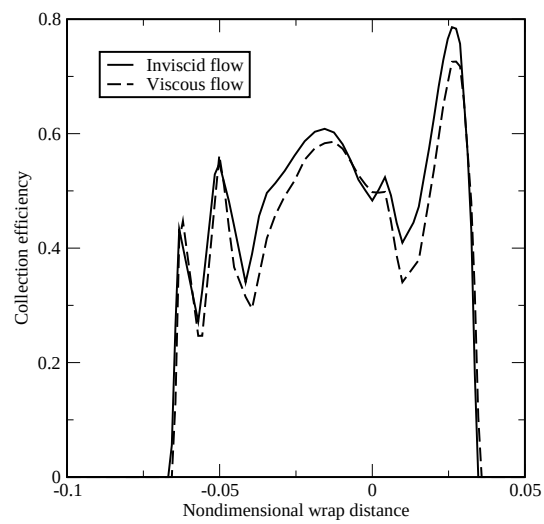


Fig. 10 Case 411: Collection efficiencies

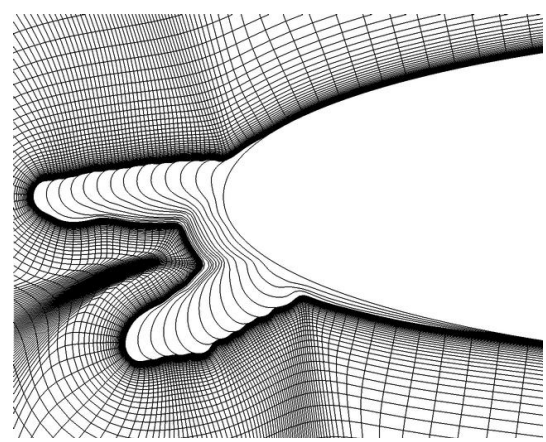


Fig. 11 Case 411: Accretion history and final grid (Level 0)

Acknowledgements

This effort was supported by the NASA Glenn Research Center under grant NAG2-2235 with Mark Potapczuk and Yung Choo as Technical Monitors. All iced airfoil geometries courtesy of NASA Glenn. We also want to acknowledge the invaluable assistance provided by William Wright of QSS Group, Inc. without whom we could not have completed this work.

References

[1]Choo, Y., Vickerman, M., Lee, K., and Thompson, D., "Geometry Modeling and Grid Generation for Icing Effects and Ice Accretion Simulations on Airfoils," *Numerical Grid Generation in Computational Fluid Simulations*, edited by B. Soni, J. Hauser, J. Thompson, and P. Eiseman, International Society for Grid Generation, Mississippi State, MS, 2000, pp. 1061–1070.

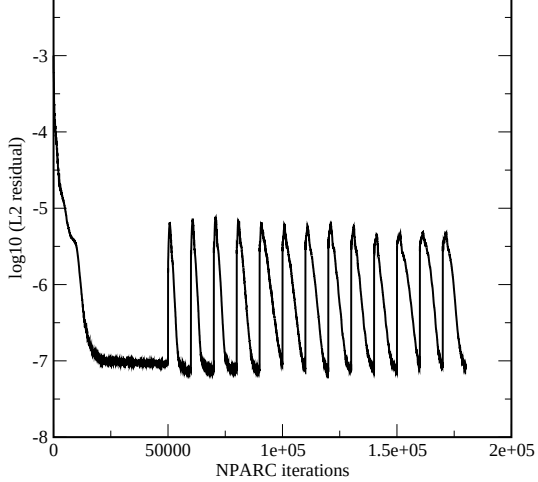


Fig. 12 Case 411: NPARC convergence history

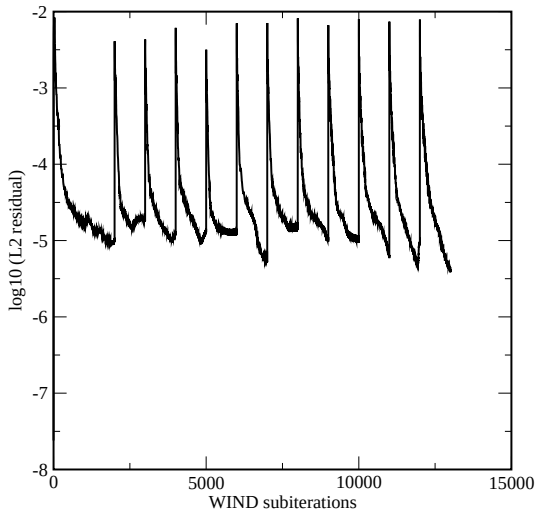


Fig. 13 Case 411: WIND convergence history

[2]Vickerman, M., Choo, Y., Braun, D., Baez, M., and Gnepp, S., "SmaggIce: Surface Modeling and Grid Generation for Iced Airfoils - Phase 1 Results," *AIAA Paper 2000-0235, AIAA 38th Aerospace Sciences Meeting*, Reno, NV, Jan. 2000.

[3]Vickerman, M., Choo, Y., Braun, D., Schilling, H., Baez, M., and Cotton, B., "Toward an Efficient Icing CFD Process using an Interactive Software Toolkit–SmaggIce 2D," *AIAA Paper 2001-0380, AIAA 39th Aerospace Sciences Meeting*, Reno, NV, Jan. 2001.

[4]Shih, T., Slater, J., and Choo, Y., "Computing Aerodynamic Performance of a 2D Airfoil: Blocking Topology and Grid Generation," *AIAA Paper 2001-0381, AIAA 39th Aerospace Sciences Meeting*, Reno, NV, Jan. 2001.

[5]Wright, W. B., "User Manual for the NASA Glenn Ice Accretion Code LEWICE (Version 2.0)," Tech. Rep. CR-1999-209409, NASA, Sept. 1999.

[6]"NPARC User's Guide Version 3.0," 1996.

[7]"WIND User's Guide," 2002.

[8]Thompson, D. S. and Soni, B. K., "ICEG2D—An Integrated Software Package for Automated Prediction of Flow Fields for Single-Element Airfoils with Ice Accretion," Tech. Rep. CR-2000-209914, NASA, Feb. 2000.

[9]Thompson, D. S. and Soni, B. K., "ICEG2D (v2.0)—An Fields for Single-Element Airfoils with Ice Accretion," Tech. Rep. CR-2001-210965, NASA, June 2001.

[10]Thompson, D. S. and Soni, B. K., "ICEG2D (v3.0)—An Integrated Software Package for Ice Accretion Prediction," Tech. rep., NASA, June 2002.

[11]"WIND Utilities Guide," 2002.

[12]"GMAN User's Guide," 2002.

[13]"CFPOST User's Guide," 2002.

[14]Soni, B., Thompson, D., Koomullil, R., and Thornburg, H., "GGTK: A Tool Kit for Static and Dynamic Geometry-Grid Generation and Adaptation," *AIAA Paper 2001-1164, AIAA 39th Aerospace Sciences Meeting*, Reno, NV, Jan. 2001.

[15]Thompson, J. F., Warsi, Z. U. A., and Mastin, W., *Numerical Grid Generation*, Elsevier Science Publishing Company, New York, 1985.

[16]Nakumura, S., "Noniterative Grid Generation Using Parabolic Partial Differential Equations," *Numerical Grid Generation*, edited by J. F. Thompson, Elsevier Science Publishing Company, Inc., New York, 1982.

[17]Noack, R. W. and Anderson, D. A., "Solution-Adaptive Grid Generation Using Parabolic Differential Equations," *AIAA J.*, Vol. 28, 1990, pp. 1016–1023.

[18]Chan, W. M. and Steger, J. L., "Enhancements of a three-dimensional grid generation scheme," *App.Math. Comp.*, Vol. 51, 1992, pp. 181–205.

[19]Thompson, D. S. and Soni, B. K., "Generation of Quad- and Hex-Dominant, Semistructured Meshes Using an Advancing Layer Scheme," *Proceedings of the 8th International Meshing Roundtable*, Oct. 1999.

[20]Thompson, D. S. and Soni, B. K., "Semistructured Grid Generation in Three-Dimensions using a Parabolic Marching Scheme," *AIAA Paper 2000-1004, AIAA 38th Aerospace Sciences Meeting*, Reno, NV, Jan. 2000.

[21]Roberts, G. O., "Computational Meshes for Boundary Layer Problems," *Lect. Notes Phys.*, Vol. 8, 1971, pp. 171–177.

[22]Soni, B. K., "Elliptic Mesh Generation Systems: Control Functions Revisited," *App. Math. Comp.*, Vol. 59, 1993, pp. 151–164.

[23]Wright, W., "private communication," 2002.

[24]Wright, W. B., "Validation Results for LEWICE 2.0," Tech. Rep. CR-1999-208690, NASA, Jan. 1999.